

Diagnostic Engine — Pre-Deploy Findings

Scope: the IELTS diagnostic flow (Listening/Reading/Writing/Speaking submission, scoring, and the one-time completion gate) at `src/controllers/diagnosticController.ts` and its dependencies.

Legend: ■ must fix before deploy · ■ should fix before deploy · ■ worth doing, not urgent · ■ low priority · ■ reviewed, no action needed

Status key: ✓ Fixed · ■ In progress (separate session) · ■ On hold (needs sign-off) · ■ Decided against — closed by design · ■ Not started

■ Must fix — real lockouts with no recovery path

1. Diagnostic save is not transactional — ✓ Fixed

When a section is submitted, the server writes to two separate tables one after another instead of as a single atomic operation. If the second write fails (a DB hiccup, timeout, or too many students submitting at once), the student ends up in a broken state: the "already submitted" check says yes, but the "section complete" check says no. She can't resubmit and can't proceed — there is no way out without someone manually fixing the database.

Fix applied: wrapped both writes in a single `prisma.$transaction`, matching the pattern already used for Mock and IA scoring.

2. Submitting with zero answers permanently locks a student out — ✓ Fixed

If a student submits a Listening or Reading section with no answers selected (e.g. connection drop, or she runs out of time with nothing filled in), the server can't figure out which question set to grade against and returns an error. The section is never marked complete, so she's stuck — there's no valid answer set for the system to reject.

Fix applied: falls back to the question set the student was actually served (validated server-side) instead of erroring out.

3. No way to reset a student's diagnostic — ✓ Fixed

There was no admin tool or API endpoint to reset a student's diagnostic if something went wrong. Only a local, untracked script existed with no safety checks.

Fix applied: admin/owner-gated endpoint to reset a student's diagnostic per skill, with a `reset_marker` so client caches invalidate correctly.

■ Should fix — real gaps, moderate effort

4. Double-submitting can grade a student twice — ✓ Fixed

Nothing stopped two submissions for the same section from being processed at the same time — e.g. a student double-taps "Submit" while the page is slow to respond.

Fix applied: Postgres advisory lock closes the race so only one submission per skill per student can be processed.

5. Writing and Speaking accept any question ID, unchecked — ■ In progress

The server currently trusts the submitted question ID without checking it belongs to the student, matches her level, or is even a Writing/Speaking question. Being worked in a separate session.

6. A server error can wrongly delete a student's recording — ✓ Fixed

When AI grading failed on our end (a real server error), the frontend treated it identically to "no speech detected," deleting a perfectly good recording and blaming the student's microphone.

Fix applied: frontend now branches on the specific error code (`no_speech_detected`) rather than the shared `can_retry` flag, so a genuine `ai_grading_failed` falls through to the existing keep-the-recording retry path. Verified via simulated 502 (recording survives, resubmit works) and regression-tested against a real no-speech case (still correctly discards and re-prompts).

7. AI grading isn't fully consistent between runs — ✓ Fixed (with a caveat)

The AI grading calls didn't fix the "creativity" setting, so the same essay resubmitted could occasionally score differently.

Fix applied: `temperature: 0` set on both Writing and Speaking Gemini calls. Verified with a real repeated-essay test — variance is significantly reduced, but Gemini does not guarantee bit-for-bit determinism even at temperature 0 (Google's own docs note floating-point/batching effects on their serving side), so identical input can still very occasionally produce a slightly different result. This residual variance is outside our code's control.

8. No retry when the AI grading service has a transient failure — ■ Closed by design

A single blip from the AI provider (rate limit, brief network issue) surfaces straight to the student as a failure requiring manual retry.

Decision: not implementing auto-retry. Since #6's fix keeps the recording/essay intact on a server-side failure, recovery is already just one manual click away — auto-retry would solve a problem that's already cheap to recover from.

9. Changing target band mid-diagnostic can change difficulty mid-test — ■ Superseded

The diagnostic assigns question difficulty based on the student's "target band," which is editable mid-diagnostic, so later sections could be served at a different difficulty than earlier ones.

Status: moot once the difficulty-disconnect plan (see below) lands — diagnostic question selection is being unhooked from `target_band` entirely, so there will be no difficulty to drift mid-test. Not being fixed independently.

10. Writing has no content-validity check; Speaking does — ■ Closed by design

Speaking has server-side logic to catch clearly invalid attempts (empty audio, off-topic, gibberish) independent of the AI. Writing has no equivalent.

Decision: not adding one — the AI prompt's own validity instructions are considered sufficiently reliable for Writing in practice.

■ Worth doing, not urgent

- **Speaking has two submit controls with no "already submitting" guard** — ■ Closed by design. #4's advisory lock already closes this race at the database level; a frontend double-click guard would be redundant.
- **The recorded transcript isn't saved to its own database column** — ■ On hold. `AssessmentHistory.transcript` is a real, dedicated column that stays NULL for diagnostic Speaking submissions (the transcript is only reachable inside the `raw_answers` JSON blob). Holding pending confirmation from Sarthak — possible the JSON-only storage is intentional (e.g. transcript size considerations).
- **Under-length Writing essays can be penalized twice** — ✓ Fixed. The overall band score was re-capped at 5.0 even though the AI prompt already caps the Task Achievement/Response criterion at 5.0 for the same reason, stacking two penalties for one shortfall. Now only the TA/TR criterion is capped server-side (guaranteed, not just trusted), and the overall band is re-derived as the mean of all four criteria — verified with a real essay where the old logic gave 5.0 and the new logic correctly gives 6.0.
- **Nothing stores the true underlying score below the 4.0 floor** — ■ Closed by design. 4.0 is an intentional floor, not a technical gap; `StudentCompetencyMatrix` is meant to stay exactly what it is — the diagnosis-derived baseline skill value, resettable via #3's admin endpoint.
- **The "is this student diagnosed?" check doesn't verify the score source** — ■ Closed by design. Same reasoning as above — `StudentCompetencyMatrix` is only ever written by the diagnosis flow, so there's no other write path today that could falsely trip completion.

■ Low priority — deferred, both acceptable to leave as-is

- **The server currently logs full request headers and bodies**, including login tokens and submitted essay/answer content. Not exploitable by a student against anyone else; risk is limited to a compromised server or leaked log file. Cheap to fix later, not urgent.
- **Microphone error messages are all collapsed into one generic message** regardless of actual cause (permission denied, no device, unsupported browser) — a UX polish item, not a correctness issue.

■ Open question — needs an answer before scoping #3's original follow-up (rate limiting)

Is rate limiting already configured at the server/proxy level (nginx, Cloudflare, etc.)? The application code itself has none, but that may already be handled outside the codebase.

■ Reviewed and closed — no action needed

- **Guessing on the 4-item diagnostic inflating scores** — self-corrects over the first few real assessments (IA), which recalibrate every skill score toward the student's true ability.

- **Diagnostic difficulty depending on a self-reported target band** — same reasoning; an inflated or deflated starting point converges to the true level within a few sessions. (Superseded in practice by the difficulty-disconnect plan below.)
 - **Pasted or AI-generated essays scoring artificially high** — same self-correction applies once real assessments begin.
 - **Client-side-only timers and typing restrictions being bypassable** — same reasoning; a student who games the diagnostic itself will still be measured accurately shortly after.
 - **New accounts being auto-created for any login** — confirmed harmless; a student record (required for literally everything else in the app) can only be created by an institute admin, so a bare account can't do anything.
 - **A single Writing/Speaking prompt instead of multiple** — an acceptable trade-off for a one-time diagnostic; not worth the added completion time for v1.
 - **Speaking scoring off-topic or mumbled answers at the minimum with no retry offered** — reviewed and accepted as correct behavior: unlike a broken recording (which does offer a retry), an off-topic or unclear answer reflects genuine performance, similar to how a real exam wouldn't offer a redo for answering the wrong question.
 - **Diagnostic completion getting the UI stuck on a spinner** — verified not reproducible; not an issue.
-

Separate, non-audit initiative in progress

Disconnecting difficulty from diagnostic question selection (per Sarthak's direction): diagnostic question sets are currently tiered by `level` (A/B/C, derived from `target_band`); the plan is to serve every student the same pool regardless of target band, then drop the now-unused `level` column once retired content is fully replaced. Code change not yet started — question-content replacement is in progress on the user's own timeline first.